

# Course ID: SW 302

## User Interface (UI) Development

Dr. Mohamed Sami Rakha

### Lecture#10

### Cont React

# Lecture Outline

Dr. Mohamed Sami Rakha

**React**

# Recap: Using state that will change! useState Hook

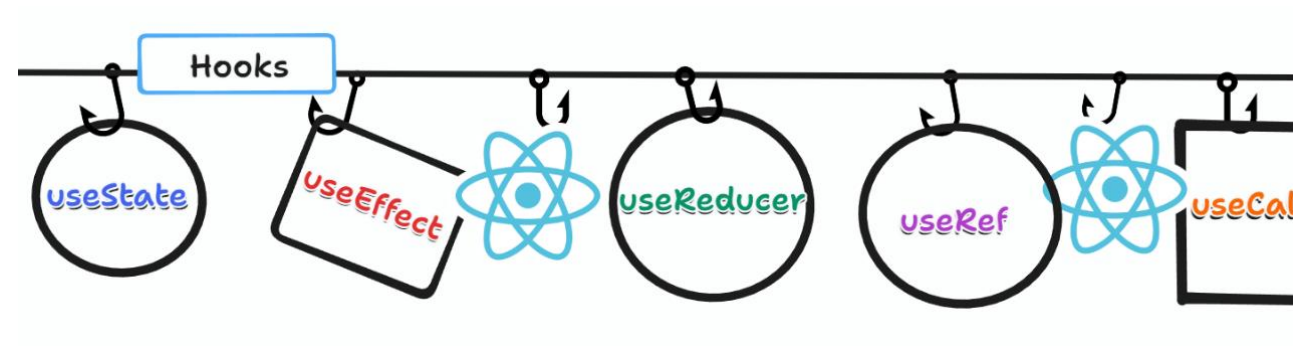
Instead of Local function variables, we use “**State Variables**” and we use the “**useState**” Hook. One of many React Hooks

**Hooks:** Special functions that allow developers to hook into **state** and **lifecycle** of React components.

**State:** One or more data values associated with a React component instance.

**Lifecycle:** The events associated with a React component instance (create, render, destroy, etc).

In React, we think of components with states. When states of a component changes, React automatically update the DOM; we almost don't need to worry about updating the DOM manually.



# Recap: React hook: useState

Purpose:

1. Remember values internally when the component re-renders
2. Tell React to re-render the component when the value changes

Syntax:

```
const [val, setVal] = useState(100);
```

The current value

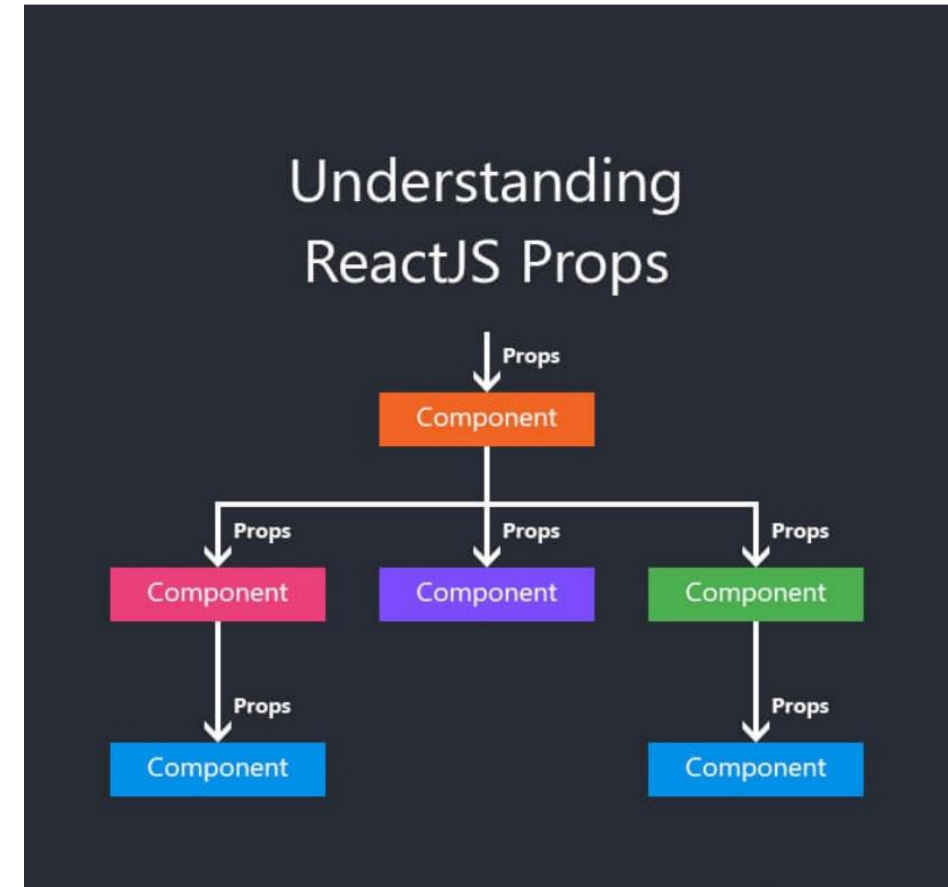
A setter function to  
change the value

The initial  
value to use

# Passing Data via Props

Dr. Mohamed Sami Rakha

- **Props** (properties) are passed to a component by its parent
  - **Values** (strings, objects, ...) to configure how the component displays or behaves
    - Top-to-bottom data flow
  - **Functions** (callbacks) to access the parent's methods
    - Bottom-to-top action requests



# Data via Props

# Passing Data via Props

Props are input to a component to make it –reusable.  
Using Props we can pass data to our components

Dr. Mohamed Sami Rakha

We need to update how we use the component “ListGroup”

```
interface Props {
  items: string[];
  heading: string;
}

function ListGroup({ items, heading }: Props) {
  const [selectedIndex, setSelectedIndex] = useState(-1);

  return (
    <>
      <h1>{heading}</h1>
      {items.length === 0 && <p>No item found</p>}
      <ul className="list-group">
        {items.map((item, index) => (
          <li
            className={
              selectedIndex === index
                ? "list-group-item active"
            }
          >{item}</li>
        ))}
      </ul>
    </>
  );
}
```

```
rc > App.tsx > App
import ListGroup from "../components/ListGroup";

function App() {
  let items = ["New York", "San Francisco", "Tokyo", "London"];

  return (
    <div>
      <ListGroup items={items} heading="Cities" />
    </div>
  );
}

export default App;
```

# Passing Function via Props

What if we want to update the Parent component or Consumer of the component about updates to happen inside? If a change happens the Parent, which is in this case the “App” component, gets notified

```
import { useState } from "react";

interface Props {
  items: string[];
  heading: string;
  onSelectItem: (item: string) => void;
}

function ListGroup({ items, heading, onSelectItem }: Props) {
  const [selectedIndex, setSelectedIndex] = useState(-1);

  return (
    <div>
      <div>
        {heading}
      </div>
      <ul>
        {items.map((item, index) => (
          <li>
            {item}
            <button>Select
          </li>
        ))}
      </ul>
    </div>
  );
}
```

adding

adding

Updating the parent component the “App”, to pass a function. We should be the function “handleSelectItem” the underlined red.

```
src > App.tsx > App
let items = ["New York", "San Francisco", "Tokyo", "London", "Paris"];

const handleSelectItem = (item: string) => {
  console.log(item);
};

return (
  <div>
    <ListGroup items={items} heading="Cities" onSelectItem={handleSelectItem} />
  </div>
);
```

adding

# What is Prop Drilling?

**Prop drilling** happens when you have to pass data through multiple nested components **just to reach a child component**, even though the intermediate components *don't use that data*.

## Example of Prop Drilling

You want to pass `user` data to the **Profile** component, but it's buried inside multiple layers:

```
<App>
  <Layout>
    <Sidebar>
      <Profile />
    </Sidebar>
  </Layout>
</App>
```

You must pass `user` through every level:

```
function App() {
  const user = { name: "Mo" };
  return <Layout user={user} />;
}

function Layout({ user }) {
  return <Sidebar user={user} />;
}

function Sidebar({ user }) {
  return <Profile user={user} />;
}

function Profile({ user }) {
  return <h3>{user.name}</h3>;
}
```

### Problem:

- **Layout and Sidebar** don't care about the `user`
- They only pass it down, making them bloated (bigger than needed)

# Props vs States

## PROPS

**Input passed to a component**

**Similar to function args**

**Immutable Design Pattern**

**Cause a re-render**

## STATE

**Data managed by a component**

**Similar to local variables**

**Mutable**

**Cause a re-render**

akha

# Immutability

- Reacts exploits **Immutability** of objects, for ease of programming and efficiency of processing
- Component '**props**' are immutable (read-only by the component)
- Component '**state**' is not directly mutable (can be changed only through special calls)
- Functions are '**pure**' (have no side-effects besides computing the return value)

# Re-Rendering

Dr. Mohamed Sami Rakha

- The application is made of Components
- The entire application is re-rendered:
  - Every time a state is changed
  - Every time a property is changed
- Each Component will re-build itself from scratch
  - With minor variations, or
  - Radically different

# Predicting component re-rendering

Dr. Mohamed Sami Rakha

A component will only re-render when...

1. A value inside **props** changes

– or –

2. A **useState** setter is called

This means all data values displayed in the HTML should depend on either **props** or **useState**

# Building a React Project

Dr. Mohamed Sami Rakha

- When you're ready to launch your app, run this command:

**npm run build**

- This bundles your app into CSS/JS/HTML files and puts them in the **/build** folder

# **Lifecycle of Components**

# Lifecycle of Components

**1. Mounting (Component is Inserted Into the DOM)** This happens **once**, when the component first appears on the screen.

## Lifecycle Methods (in order)

### (a) constructor()

Initializes state.

Binds methods.

Runs **before the component is mounted**.

### (b) getDerivedStateFromProps()

Rarely used.

Syncs state with incoming props

Runs **before every render** (mount + update).

### (c) render()

**Pure function** → should NOT have side effects (no DOM changes, no fetch).

Returns JSX to display UI.

### (d) componentDidMount()

Runs **after the component is added to the DOM**.

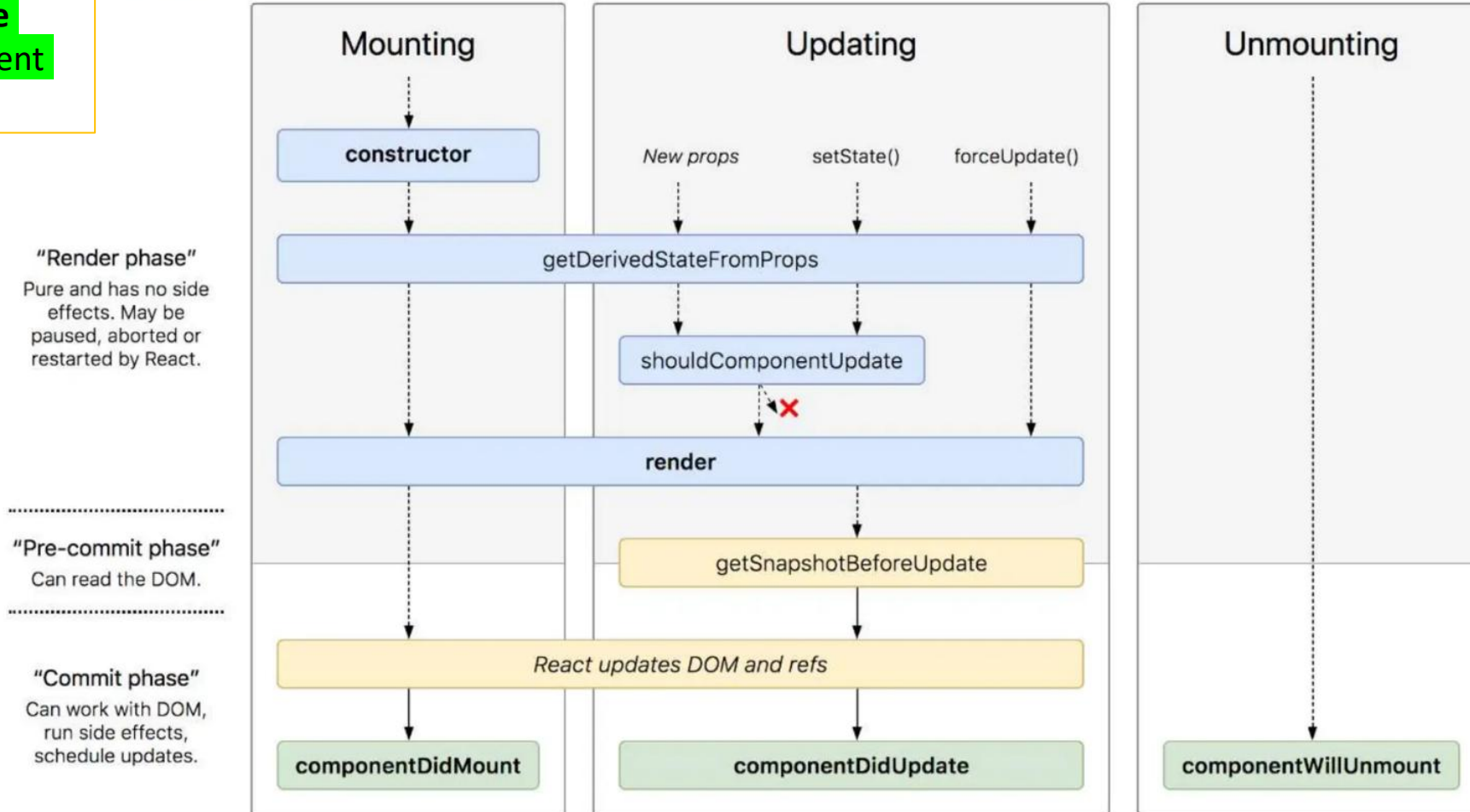
Perfect for:

- Fetching data (API calls)

- Subscribing to events

- Setting up timers

Only runs **once**.



# Lifecycle of Components

## 2. Updating (Component Re-renders due to State or Props Changes)

### Lifecycle Methods (in order)

#### (a) `getDerivedStateFromProps()`

Same method as in mounting.

#### (b) `shouldComponentUpdate()`

Determines whether the component **should re-render**.

Used for performance optimization.

Returning false stops the update.

#### (c) `render()`

Called again to update UI.

#### (d) `getSnapshotBeforeUpdate()`

Runs **just before React updates the DOM**.

Used to capture scroll position, cursor position, etc.

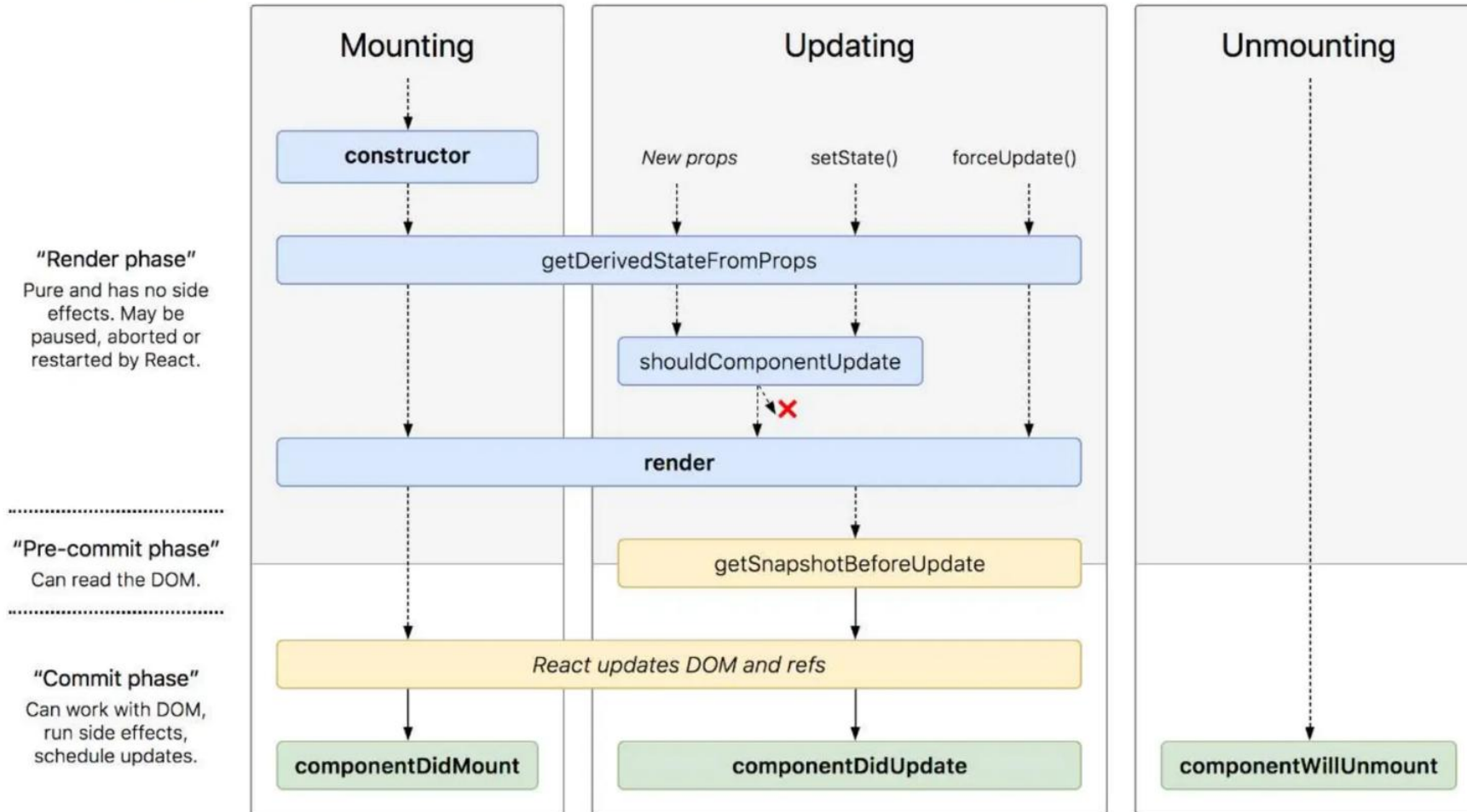
Value returned here is passed to `componentDidUpdate`.

#### (e) `componentDidUpdate()`

Runs **after DOM updates**.

Safe place to: Perform side effects, Update DOM manually, Re-fetch/update based on changed props or state

Must avoid infinite loops.



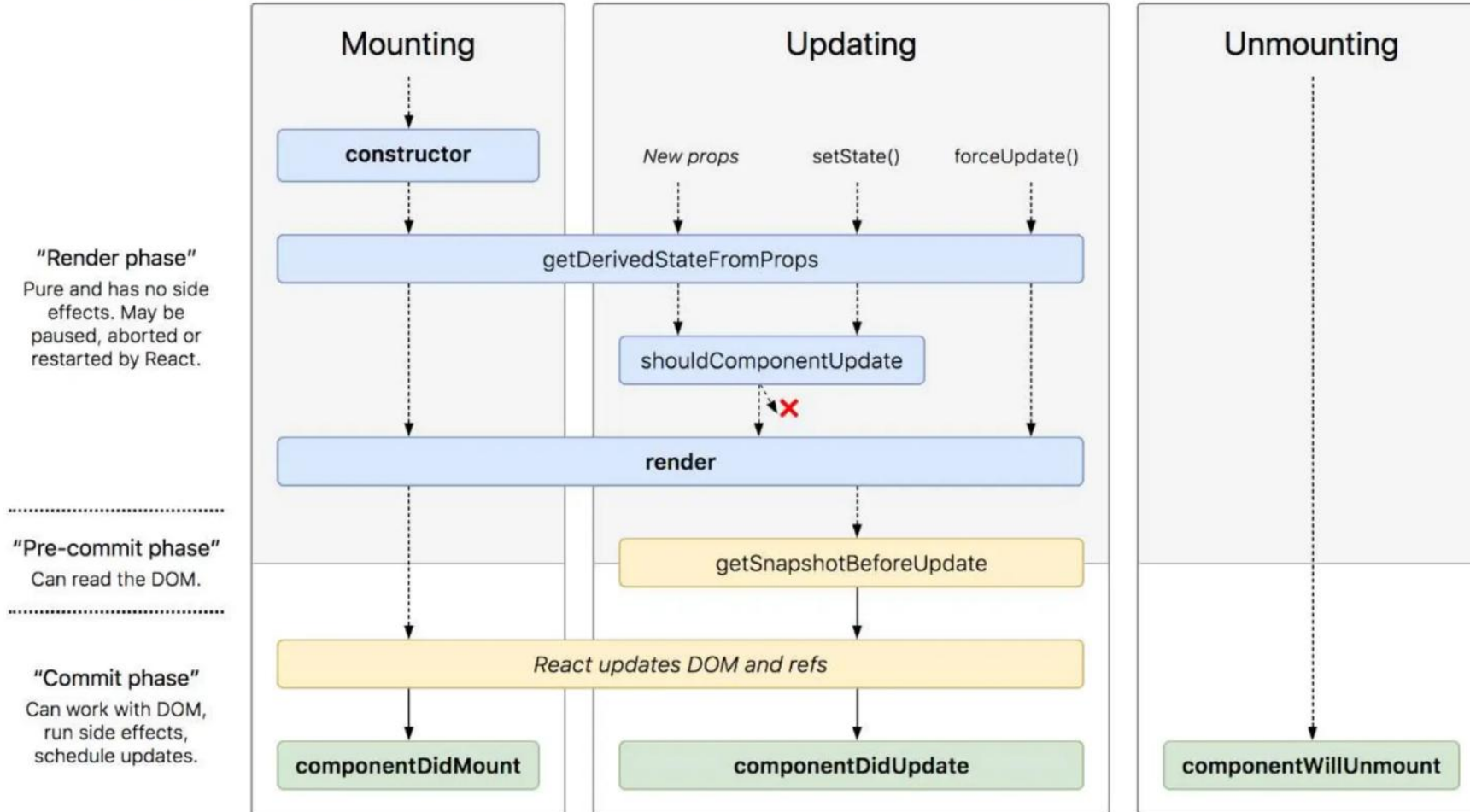
# Lifecycle of Components

## 3. Unmounting (Component Removed From DOM)

**componentWillUnmount()**

**Runs just before the component disappears.**

- Ideal for cleanup:
- Remove event listeners
- Clear timers
- Cancel API calls
- Unsubscribe from external services



# React Router

# What is React Router?

**React Router** is a client-side routing library for React that allows you to build single-page applications with multiple “pages” without reloading the browser. **React Router is an external library**, not part of core React. React itself **does not include routing** (on purpose). React focuses only on the **UI layer**.

It changes which component is shown based on the URL, without a full-page *refresh*.

## Why Do We Need React Router?

React by default shows **one page/component**

But real apps need:

- /login
- /dashboard
- /products/123
- /settings

**React Router** lets us navigate between “pages” smoothly  
Keeps SPA **[Single Page Application]** experience: fast, no reloads, keeps state

### 1] Installing React Router:

```
npm install react-router-dom
```

### 2] You must wrap the app in a **BrowserRouter**:

```
import { BrowserRouter } from "react-router-dom";  
import App from "./App";  
  
root.render(  
  <BrowserRouter>  
    <App />  
  </BrowserRouter>  
);
```

# Why React Router Is Helpful?

## 1- Enables Single-Page Applications (SPA)

React Router lets you build **multi-page experiences** inside a **single HTML file**.  
No full page reloads → smoother, faster, app-like user experience.

## 2. Manages Navigation Without Reloading

Traditional websites reload the entire page when navigating.  
React Router updates the view using **JavaScript routing**, so only the necessary components change.

## 3. Creates Cleaner, Organized Code Structure

It lets you break your UI into logical pages like:

- /login
- /dashboard
- /products/:id

## 4. Supports Dynamic URLs

You can create routes with parameters:

/user/123

/product/abc

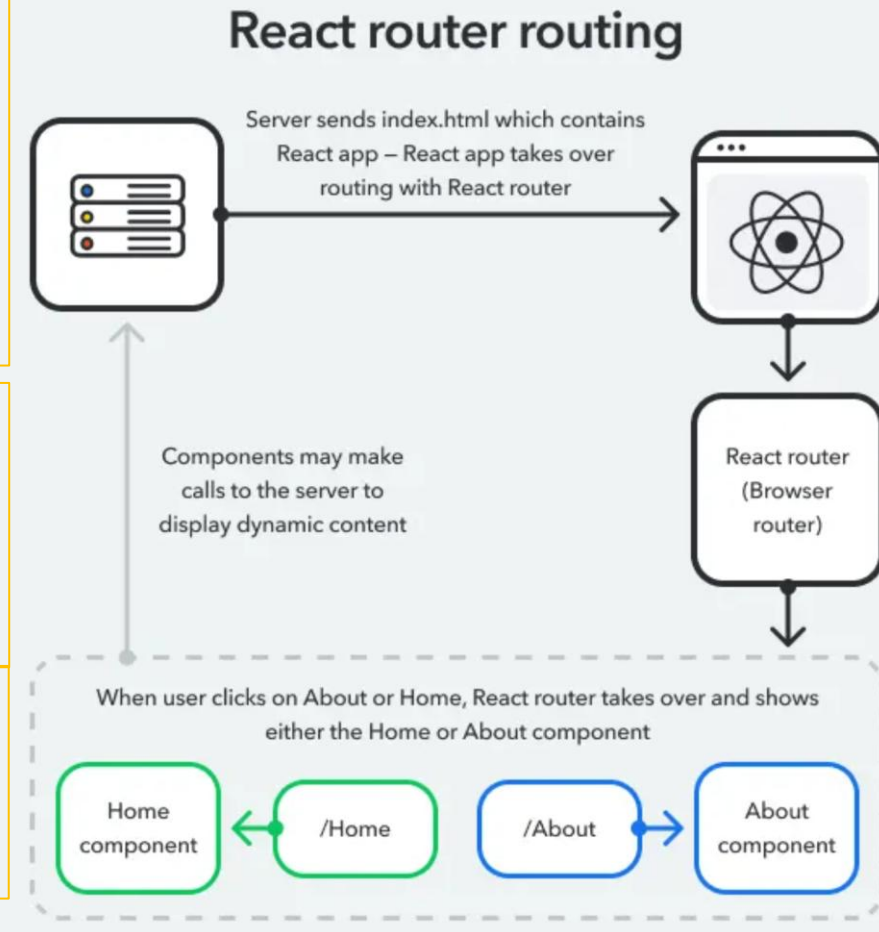
React Router extracts those params and passes them to your components.

## 5. Essential for Real Applications

Most real systems need pages such as:

Login, Dashboard, Profile, Settings, Product details

**Without routing, everything would be one messy component.**



# Basic Example (React Router)

This example shows how page loads on URL, how `<BrowserRouter>`, `<Routes>`, and `<Route>` work. The minimal setup for navigation in a React app.

## Create simple pages:

```
// Home.jsx
export default function Home() {
  return <h1>Home Page</h1>;
}

// About.jsx
export default function About() {
  return <h1>About Page</h1>;
}
```

## Configure Routes:

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "./Home";
import About from "./About";

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

# **More on React Hooks**

# Classification of React Hooks

## Basic Hooks

**useState** → Manages state

**useEffect** → Side effects (fetching, timers, subscriptions)

**useContext** → Share global data

## Additional Hooks

**useReducer** → Alternative to useState for complex logic

**useCallback** → Memoize functions

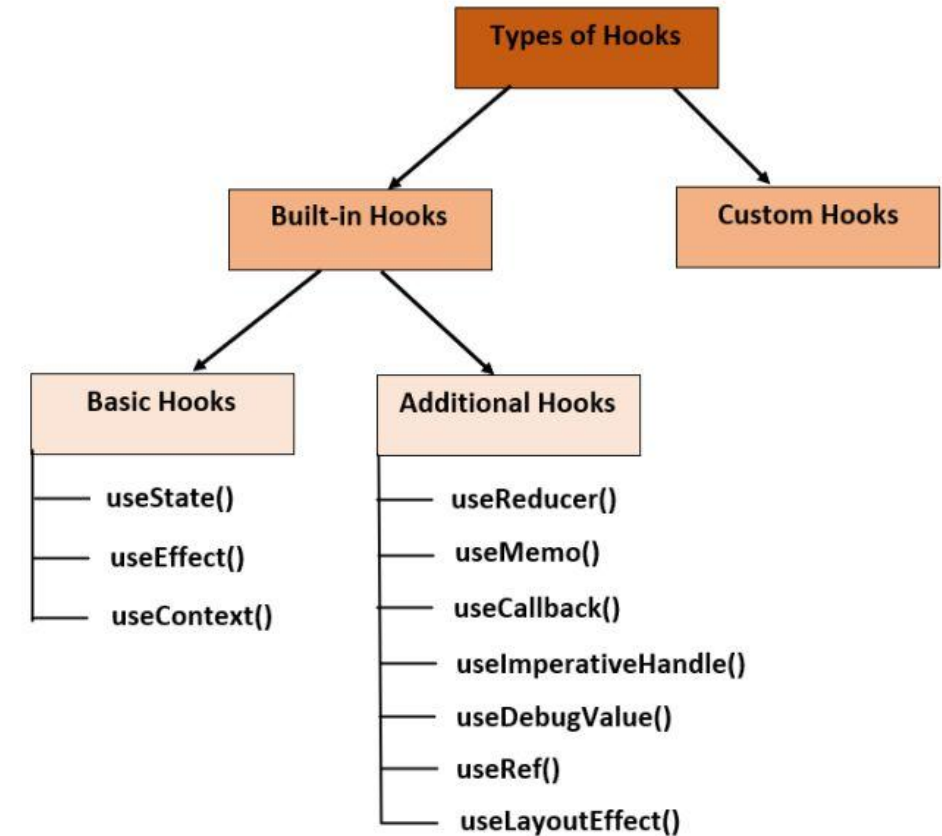
**useMemo** → Memoize values

**useRef** → DOM access & persistent values

**useImperativeHandle** → Customize ref behavior

**useLayoutEffect** → Runs before browser paints

**useDebugValue** → Show custom labels in React DevTools



# React hook: useEffect

`useEffect` is a React Hook that lets you **run side effects** in your components. A *side effect* is anything that affects something **outside the component** or needs code to run **after rendering**.

Purpose:

Act as an **observer**, running code in response to value changes

Syntax:

```
useEffect(() => {  
  console.log(`myValue was changed! New value: ${myValue}`);  
}, [myValue]);
```



A list of values such that changes  
should trigger this code to run



The code to run when  
values change

# Why do we need useEffect?

**React renders UI → then** you sometimes need to do something *after* the UI updates, such as:

- Fetching data from an API
- Updating the document title
- Subscribing/unsubscribing to events
- Setting timers (setTimeout, setInterval)
- Storing data in localStorage

You can control **when** your effect runs.

## A) Run Once (on component mount)

```
useEffect(() => {  
  console.log("Runs only once");  
}, []);
```

## B) Run When State or Props Change

```
useEffect(() => {  
  console.log("Runs when count changes");  
}, [count]);
```

## C) Run After Every Render

```
useEffect(() => {  
  console.log("Runs every time");  
});
```

# React lifecycle methods and their equivalents in React Hooks

| React Lifecycle Methods                             | React Hooks                                                                                                                                                                                                                          |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>componentDidMount() {<br/>  ...<br/>}</pre>    | <pre>useEffect(() =&gt; {<br/>  // called once after first rendering<br/>  // due to empty array as second argument<br/>}, []);</pre>                                                                                                |
| <pre>componentWillUnmount() {<br/>  ...<br/>}</pre> | <pre>useEffect(() =&gt; {<br/>  ...<br/>  return () =&gt; {<br/>    // called when unmounting<br/>  }<br/>}, []);</pre>                                                                                                              |
| <pre>componentDidUpdate() {<br/>  ...<br/>}</pre>   | <pre>useEffect(() =&gt; {<br/>  // called after every rendering<br/>  // since there's no second argument<br/>});<br/><br/>useEffect(() =&gt; {<br/>  // called only when dependencies<br/>  // a and b change<br/>}, [a, b]);</pre> |

mi Rakha

# useEffect Examples

| Hook                   | Purpose                                     |
|------------------------|---------------------------------------------|
| <code>useState</code>  | Store and update data inside the component  |
| <code>useEffect</code> | Run code after React renders (side effects) |

## API Fetching

```
useEffect(() => {  
  fetch("https://api.example.com/users")  
    .then(response => response.json())  
    .then(data => setUsers(data));  
}, []);
```

## Updating the Document Title

```
useEffect(() => {  
  document.title = `Count: ${count}`;  
}, [count]);
```

## Listening for Keyboard Events

```
useEffect(() => {  
  const handleKey = (e) => console.log(e.key);  
  
  window.addEventListener("keydown", handleKey);  
  
  return () => window.removeEventListener("keydown", handleKey);  
}, []);
```

In this example, This `useEffect` sets up a **global keyboard listener** that logs every key the user presses. It runs **once** when the component mounts and removes the listener when the component unmounts.

# React + Tailwind + useEffect Example (Static Cities)

## This example includes:

- Tailwind styling
- useEffect running on mount
- useEffect reacting to selected city
- Highlighting selected city

```
import React, { useState, useEffect } from "react";

export default function ListGroup() {
  const cities = ["Cairo", "Alexandria", "Giza", "Luxor", "Aswan"];
  const [selectedCity, setSelectedCity] = useState("");

  // Runs once when component loads
  useEffect(() => {
    console.log("ListGroup mounted");
  }, []);

  // Runs every time the selected city changes
  useEffect(() => {
    if (selectedCity) {
      document.title = `Selected: ${selectedCity}`;
    }
  }, [selectedCity]);
}
```

Same  
Component



```
return (
  <div className="p-6 max-w-md mx-auto bg-white rounded-xl shadow-md">
    <h2 className="text-xl font-semibold mb-4">Cities</h2>

    <ul className="space-y-2">
      {cities.map((city) => (
        <li
          key={city}
          onClick={() => setSelectedCity(city)}
          className={`p-3 rounded cursor-pointer border transition
            ${
              selectedCity === city
                ? "bg-blue-500 text-white border-blue-600"
                : "bg-gray-100 hover:bg-gray-200 border-gray-300"
            }
          `}
        >
          {city}
        </li>
      ))}
    </ul>

    {selectedCity && (
      <p className="mt-4 text-gray-700">
        You selected: <span className="font-bold">{selectedCity}</span>
      </p>
    )}
  </div>
);
```



# React + Tailwind + useEffect Example (API Fetch Version)

This is the “real” useEffect example, fetching cities from an API.

```
import React, { useState, useEffect } from "react";

export default function ListGroup() {
  const [cities, setCities] = useState([]);
  const [loading, setLoading] = useState(true);

  // Fetch cities when component loads
  useEffect(() => {
    fetch("https://countriesnow.space/api/v0.1/countries/cities?q=egypt")
      .then((res) => res.json())
      .then((data) => {
        setCities(data.data);
        setLoading(false);
      })
      .catch((err) => {
        console.error(err);
        setLoading(false);
      });
  }, []);
```

Same  
Component

```
return (
  <div className="p-6 max-w-md mx-auto bg-white rounded-xl shadow-md">
    <h2 className="text-xl font-semibold mb-4">Cities</h2>

    {loading ? (
      <p className="text-gray-500">Loading cities...</p>
    ) : (
      <ul className="space-y-2">
        {cities.map((city) => (
          <li
            key={city}
            className="p-3 rounded bg-gray-100 border border-gray-300 hover:
          >
            {city}
          </li>
        ))}
      </ul>
    )}
  </div>
);
```

# useContext Hook

**useContext** is a React hook that allows a component to access values **directly** from a shared context **without passing props manually through every level** of the component tree.

It solves the problem of "**prop drilling**", passing props through many components just so a deeply nested component can use them.

## 1. Create a Context

You create a context using:

```
const MyContext = React.createContext();
```

This creates a "storage box" where shared data will live.

## 2. Wrap Components with a Provider

The Provider makes the data available to any component inside it:

```
<MyContext.Provider value={{ user: "Mo" }}>
  <App />
</MyContext.Provider>
```

- Any component inside <MyContext.Provider> can access the value.
- You can store anything: strings, objects, arrays, functions, etc.

## 3. Use the Context in a Component

Inside any child component, use the hook:

```
const data = useContext(MyContext);
```

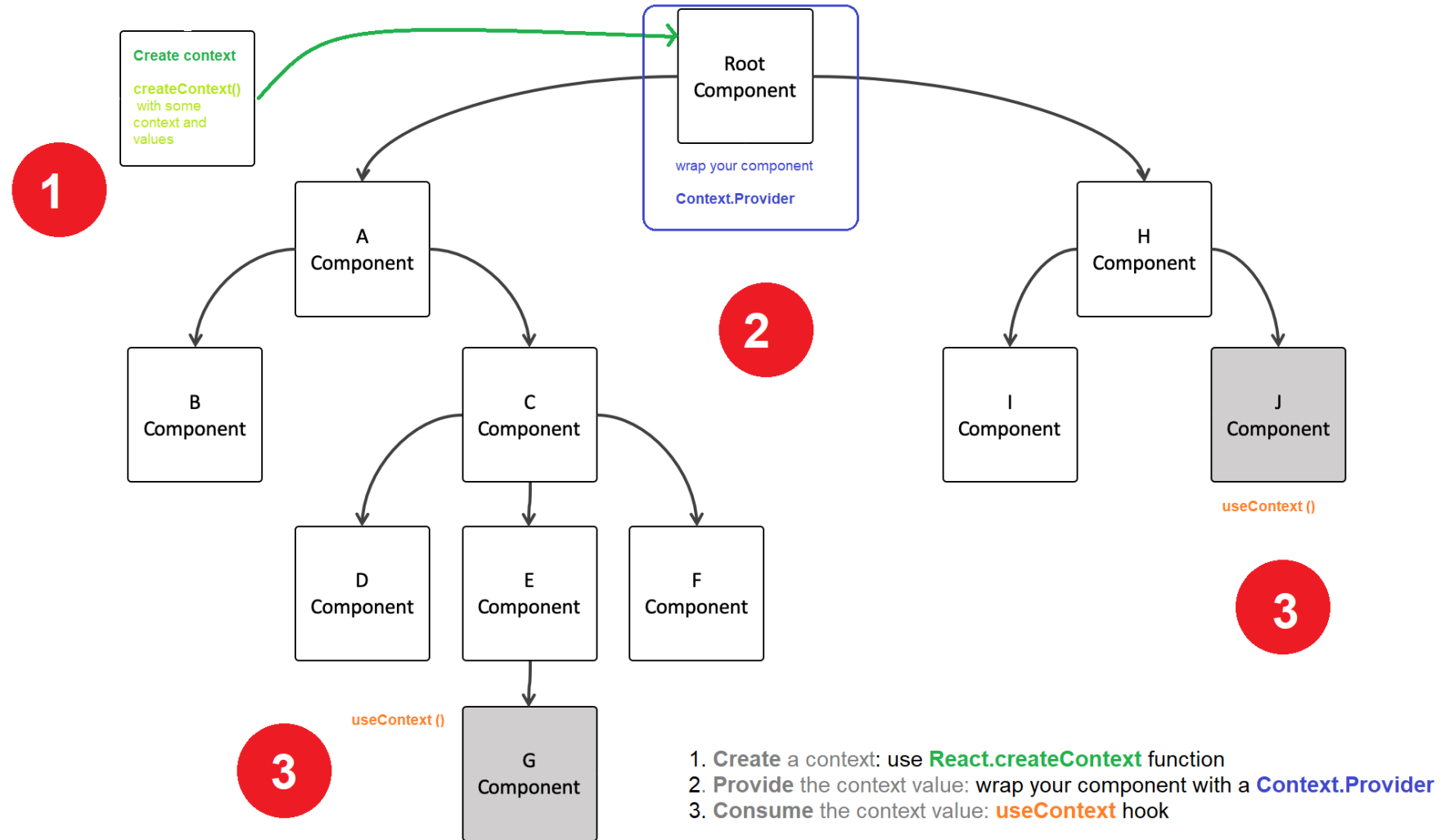
```
function Profile() {
  const { user } = useContext(MyContext);
  return <h1>Hello, {user}</h1>;
}
```

Now `data` contains whatever you placed in the Provider value.

## When to Use **useContext**?

- ✓ You have global data
- ✓ Many components need the same data
- ✓ You want to avoid prop drilling

# useContext Hook



# useContext Hook: Example

In this example, the `MyContext` context is created using the `React.createContext` function, with a default value of `default value`. The `MyComponent` component consumes the context value using the `useContext` hook, and **displays the value in the DOM**.

App.js

```
1  const MyContext = React.createContext('default value');
2
3  function MyComponent() {
4    const value = useContext(MyContext);
5    return <div>{value}</div>;
6  }
7
8  function App() {
9    return (
10     <MyContext.Provider value="Hello, world!">
11       <MyComponent />
12     </MyContext.Provider>
13   );
14 }
```

Dr. Mohamed Sami Rakha  
`useContext(SomeContext)`

```
1  import { useContext } from 'react';
2
3  function MyComponent() {
4    const theme = useContext(ThemeContext);
5    // ..
```

- `useContext` returns the context value for the calling component. It is determined as the value passed to the closest `SomeContext.Provider` above the calling component in the tree.

If there is no such `provider`, then the returned value will be the `defaultValue` you have passed to `createContext` for that `context`. **The returned value is always up-to-date. React automatically re-renders components that read some context if it changes.**

Dr. Mohamed Sami Rakha

**Thank You...**